01_cnn_bilstm_inceptionv3

November 3, 2026

1 CNN (InceptionV3) + BiLSTM — Image Captioning sur Flickr30k

Prérequis : `data/Images/` et `data/captions.txt` (voir `data/README.md`). Les artefacts (`tokenizer.pkl`, `model_checkpoint.h5`) sont écrits dans `models/`. Lancer ce notebook depuis le dossier `notebooks/`.

2 biblio

[5]:
```
import os
import numpy as np
from PIL import Image
import string
import pickle
import tensorflow as tf
from tensorflow.keras.applications.inception_v3 import InceptionV3,␣
 ↪preprocess_input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.layers import Input, Dense, Dropout,␣
 ↪BatchNormalization,LSTM,Embedding

from tqdm.notebook import tqdm
```

[6]:
```
DATA_DIR = "../data"
MODELS_DIR = "../models"
os.makedirs(MODELS_DIR, exist_ok=True)
file = os.path.join(DATA_DIR, "Images")
```

3 Extract Image Features

[7]:
```
base_model = InceptionV3()
model = Model(inputs=base_model.input, outputs=base_model.layers[-2].output)
```

[4]:
```
features = {}
directory = os.path.join(file)

for img_name in tqdm(os.listdir(directory)):
    # join the directory path and image name
    img_path = os.path.join(directory, img_name)

    # check if it's a file (not a subdirectory)
    if os.path.isfile(img_path):
        try:
            # load image from file
            image = load_img(img_path, target_size=(299, 299))
            # convert image pixels to numpy array
            image = img_to_array(image)
            image = image.reshape((1, image.shape[0], image.shape[1], image.
 ↪shape[2]))
            # preprocess image
            image = preprocess_input(image)
            feature = model.predict(image, verbose=0)
            # get image id
            image_id = img_name.split('.')[0]
            # store feature
            features[image_id] = feature
        except Exception as e:
            print(f"Error processing {img_path}: {e}")

with open(os.path.join(MODELS_DIR,'features1.pkl'),'wb') as f:
    pickle.dump(features,f)

# Now 'features' should contain the extracted features for each valid image
```

```
  0%|          | 0/31783 [00:00<?, ?it/s]
```

[8]:
```
# load features
with open(os.path.join(MODELS_DIR,'features1.pkl'),'rb') as f:
    features = pickle.load(f)
```

4 Perform Data Cleaning

[9]:
```
# Lire captions.txt proprement sans os
with open(os.path.join(DATA_DIR, "captions.txt"), 'r', encoding='utf-8') as f:
    next(f)  # Sauter la première ligne si besoin
    captions_doc = f.read()
```

5 Cleaning and mapping

[10]:
```
# create mapping of image to captions
mapping = {}
# process lines
for line in tqdm(captions_doc.split('\n')):
    # split the line by comma(,)
    tokens = line.split(',')
    if len(line) < 2:
        continue
    image_id, caption = tokens[0], tokens[1:]
    # remove extension from image ID
    image_id = image_id.split('.')[0]
    # convert caption list to string
    caption = " ".join(caption)
    # create list if needed
    if image_id not in mapping:
        mapping[image_id] = []
    # store the caption
    mapping[image_id].append(caption)
```

```
  0%|          | 0/158916 [00:00<?, ?it/s]
```

[11]:
```
len(mapping)
```

[11]:
```
31783
```

6 Preprocess Text

[12]:
```
def clean(mapping):
    for key, captions in mapping.items():
        for i in range(len(captions)):
            # take one caption at a time
            caption = captions[i]
            # preprocessing steps
            # convert to lowercase
            caption = caption.lower()
            # delete digits, special chars, etc.,
            caption = caption.replace('[^A-Za-z]', '')
```

```
            # delete additional spaces
            caption = caption.replace('\s+', ' ')
            # add start and end tags to the caption
            caption = 'startseq ' + " ".join([word for word in caption.split()␣
 ↪if len(word)>1]) + ' endseq'
            captions[i] = caption
```

[13]:
```
# process text
clean(mapping)
```

[14]:
```
all_captions = []
for key in mapping:
    for caption in mapping[key]:
        all_captions.append(caption)
```

[15]:
```
len(all_captions)
```

[15]:
```
158915
```

[16]:
```
all_captions[:10]
```

[16]:
```
['startseq two young guys with shaggy hair look at their hands while hanging out
in the yard endseq',
 'startseq two young white males are outside near many bushes endseq',
 'startseq two men in green shirts are standing in yard endseq',
 'startseq man in blue shirt standing in garden endseq',
 'startseq two friends enjoy time spent together endseq',
 'startseq several men in hard hats are operating giant pulley system endseq',
 'startseq workers look down from up above on piece of equipment endseq',
 'startseq two men working on machine wearing hard hats endseq',
 'startseq four men on top of tall structure endseq',
 'startseq three men on large rig endseq']
```

7 Tokenize the content

[17]:
```
# tokenize the text
tokenizer = Tokenizer()
tokenizer.fit_on_texts(all_captions)
vocab_size = len(tokenizer.word_index) + 1
```

[18]:
```
with open(os.path.join(MODELS_DIR,'tokenizer.pkl'), 'wb') as f:
    pickle.dump(tokenizer, f)
```

[19]:
```
vocab_size
```

[19]:
```
18319
```

[20]:
```
# get maximum len of the captions available
max_length = max(len(caption.split()) for caption in all_captions)
max_length
```

[20]:
```
74
```

8 Train Test Split

[21]:
```
image_ids = list(mapping.keys())
split = int(len(image_ids)  * 0.90)
train = image_ids[:split]
test = image_ids[split:]
```

9 Create Data Generator

[22]:
```
def data_generator(data_keys, mapping, features, tokenizer, max_length,␣
 ↪vocab_size, batch_size):
    # Loop over images
    x1, x2, y = list(), list(), list()
    n = 0
    while 1:
        for key in data_keys:
            n += 1
            captions = mapping[key]
            # process each caption
            for caption in captions:
                # encode the sequence
                seq = tokenizer.texts_to_sequences([caption])[0]
                # Sllt the squences into x,y pairs
                for i in range(1, len(seq)):
                    # split into input and output pairs
                    in_seq, out_seq = seq[:i], seq[i]
                    # pad input sequence
                    in_seq  = pad_sequences([in_seq], maxlen=max_length)[0]
                    # encode output sequence
                    out_seq = to_categorical([out_seq],␣
 ↪num_classes=vocab_size)[0]

                    # store the sequeces
                    x1.append(features[key][0])
                    x2.append(in_seq)
                    y.append(out_seq)

                if n == batch_size:
                    x1,x2,y = np.array(x1), np.array(x2), np.array(y)
                    yield[x1,x2], y
```

```
                    x1, x2, y = list(), list(), list()
                    n = 0
```

10 Model Architecture

[23]:
```
import gc
import numpy as np
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Bidirectional,␣
 ↪LSTM,BatchNormalization,Concatenate


# Assuming you have defined vocab_size and max_length

# Encoder model
inputs1 = Input(shape=(2048,))
fe1 = BatchNormalization()(inputs1)
fe2 = Dense(512, activation='relu')(fe1)

inputs2 = Input(shape=(max_length,))
se1 = Embedding(vocab_size, 512, mask_zero=True)(inputs2)
se2 = BatchNormalization()(se1)
se3 = Bidirectional(LSTM(256))(se2)

# Decoder
decoder = Concatenate()([fe2, se3])
decoder2 = Dense(512, activation='relu')(decoder)
outputs = Dense(vocab_size, activation='softmax')(decoder2)

model = Model(inputs=[inputs1, inputs2], outputs=outputs)
optimizer = Adam(learning_rate=0.0001, clipvalue=5.0)
model.compile(loss='categorical_crossentropy', optimizer=optimizer)
```

[24]:
```
epochs = 20
batch_size = 16

steps_per_epoch = len(train) // batch_size

checkpoint_filepath = os.path.join(MODELS_DIR, 'model_checkpoint.h5')
model_checkpoint_callback = ModelCheckpoint(
    filepath=checkpoint_filepath,
    save_weights_only=False,
    monitor='loss',
    mode='min',
```

```
    save_best_only=True
)

# Créer une seule fois le generator
generator = data_generator(train, mapping, features, tokenizer, max_length,␣
 ↪vocab_size, batch_size)

# Entraîner sur plusieurs epochs directement
history = model.fit(
    generator,
    epochs=epochs,
    steps_per_epoch=steps_per_epoch,
    verbose=1,
    callbacks=[model_checkpoint_callback]
)
```

```
Epoch 1/20
1787/1787 [==============================] - 1689s 943ms/step - loss: 4.7349
Epoch 2/20
1787/1787 [==============================] - 1345s 752ms/step - loss: 3.7951
Epoch 3/20
1787/1787 [==============================] - 1343s 752ms/step - loss: 3.4735
Epoch 4/20
1787/1787 [==============================] - 1775s 993ms/step - loss: 3.2554
Epoch 5/20
1787/1787 [==============================] - 1733s 970ms/step - loss: 3.0793
Epoch 6/20
1787/1787 [==============================] - 2141s 1s/step - loss: 2.9332
Epoch 7/20
1787/1787 [==============================] - 2202s 1s/step - loss: 2.8142
Epoch 8/20
1787/1787 [==============================] - 1951s 1s/step - loss: 2.7154
Epoch 9/20
1787/1787 [==============================] - 2252s 1s/step - loss: 2.6245
Epoch 10/20
1787/1787 [==============================] - 1357s 759ms/step - loss: 2.5410
Epoch 11/20
1787/1787 [==============================] - 1355s 758ms/step - loss: 2.4642
Epoch 12/20
1787/1787 [==============================] - 1674s 937ms/step - loss: 2.3926
Epoch 13/20
1787/1787 [==============================] - 1822s 1s/step - loss: 2.3230
Epoch 14/20
1787/1787 [==============================] - 1380s 772ms/step - loss: 2.2581
Epoch 15/20
1787/1787 [==============================] - 1338s 749ms/step - loss: 2.1974
Epoch 16/20
1787/1787 [==============================] - 1391s 778ms/step - loss: 2.1406
```

```
Epoch 17/20
1787/1787 [==============================] - 1766s 988ms/step - loss: 2.0849
Epoch 18/20
1787/1787 [==============================] - 2633s 1s/step - loss: 2.0324
Epoch 19/20
 107/1787 [>…] - ETA: 28:22 - loss: 2.0072
```

[25]:
```
model = load_model(os.path.join(MODELS_DIR, 'model_checkpoint.h5'))
```

11 Generate Captions For Images

[26]:
```
def idx_to_word(integer, tokenizer):
    for word, index in tokenizer.word_index.items():
        if index == integer:
            return word
    return None
```

[27]:
```
def predict_caption(model, image, tokenizer, max_length):
    # add start tag for generation process
    in_text = 'startseq'
    # iterate over the max length of sequence
    for i in range(max_length):
        # encode input sequence
        sequence = tokenizer.texts_to_sequences([in_text])[0]
        # pad the sequence
        sequence = pad_sequences([sequence], max_length)
        # predict next word
        yhat = model.predict([image, sequence], verbose=0)
        # get index with high probability
        yhat = np.argmax(yhat)
        # convert index to word
        word = idx_to_word(yhat, tokenizer)
        # stop if word not found
        if word is None:
            break
        # append word as input for generating next word
        in_text += " " + word
        # stop if we reach end tag
        if word == 'endseq':
            break

    return in_text
```

[44]:
```
from PIL import Image
import matplotlib.pyplot as plt
def generate_caption(image_name):
    # load the image
```

```
    # image_name = "1001773457_577c3a7d70.jpg"
    image_id = image_name.split('.')[0]
   #img_path = os.path.join(file, image_name)
   #image = Image.open(img_path)
   #captions = mapping[image_id]
   #print('---------------------Actual---------------------')
   # for caption in captions:
   #     print(caption)
   # # predict the caption
    y_pred = predict_caption(model, features[image_id], tokenizer, max_length)
    print('--------------------Predicted--------------------')
    print(y_pred)
    plt.imshow(image)
```

[69]:
```
inception_model = InceptionV3()
# restructure the model
inception_model = Model(inputs=inception_model.inputs, outputs=inception_model.
 ↪layers[-2].output)
#using real images
image_path = "../static/uploads/example.jpg"  # any test image
# load image
image = load_img(image_path, target_size=(299, 299))
# convert image pixels to numpy array
image = img_to_array(image)
# reshape data for model
image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
# preprocess image for vgg
image = preprocess_input(image)
# extract features
feature = inception_model.predict(image, verbose=0)
# predict from the trained model
predict_caption(model, feature, tokenizer, max_length)
```

[69]:
```
'startseq two girls are running in race endseq'
```

[ ]:

[ ]:

[ ]: